

Competitive Programming

Section 31: Sieve, Primes, and Factorization

Competitive Programming Course

2026-08-11

Table of contents

1	Why preprocessing wins	3
2	Vocabulary and the core invariant	4
3	Sieve of Eratosthenes	5
3.1	Trace: sieving through 20	5
4	Prime range queries: turn flags into counts	6
4.1	Trace: a prefix answer	6
5	Smallest prime factors	7
5.1	Trace: factor 360	7
6	Divisor counts from exponents	8
7	Implementation notes	9
8	Wrong turns to reject early	10
9	Formative checks	11
10	Local judge workflow	12
11	Submission checklist	13
12	Exercises	14

1 Why preprocessing wins

Trial division is a good tool for one modest integer. It is the wrong default when a statement gives up to 200,000 values, all at most 10^6 . Rather than rediscovering factors for every query, do one pass over the whole value range.

Use this section's tools when the statement says one or more of:

Signal in the statement	First tool
many interval queries about primes	sieve plus prefix counts
many values must be factorized	smallest-prime-factor (SPF) table
divisor count, distinct factors, or exponents	SPF, then group equal factors
one number near 10^{12} only	trial division may still be preferable

The local problems have maximum value 10^6 , so arrays of size 10^6+1 are intentional: they turn expensive repeated work into constant-time lookup.

2 Vocabulary and the core invariant

A **prime** has exactly two positive divisors, 1 and itself. A **composite** number has a nontrivial factor. Every integer $x > 1$ has a unique prime factorization, for example:

$$360 = 2^3 * 3^2 * 5$$

The sieve maintains this invariant after processing a candidate p :

Every composite number below $p*p$ has already been marked non-prime.

That is why marking starts at $p*p$: a smaller multiple $p*t$ has $t < p$ and was already handled when its smaller factor was processed.

3 Sieve of Eratosthenes

Start by marking every index from 2 as possibly prime; explicitly set 0 and 1 to false. For each still-prime p , mark its multiples from $p*p$.

```
set is_prime[x] = true for every x from 0 to N
is_prime[0] = is_prime[1] = false
for p from 2 while p*p <= N:
    if is_prime[p]:
        for x from p*p to N step p:
            is_prime[x] = false
```

Thus `is_prime[2]` is set to true by the first line. It remains true because the marking loop for $p=2$ starts at $2*2=4$; the prime itself is never marked false. More generally, a number remains true exactly when no smaller prime divisor marks it as composite.

The time is $O(N \log \log N)$ and the memory is $O(N)$. For $N = 10^6$, this is comfortably small in both C++ and Python.

3.1 Trace: sieving through 20

Initially, treat 2..20 as prime candidates. Processing 2 crosses out:

```
4, 6, 8, 10, 12, 14, 16, 18, 20
```

Processing 3 starts at 9, not 6, and crosses out:

```
9, 12, 15, 18
```

The numbers still true are 2, 3, 5, 7, 11, 13, 17, 19. When $p=5$, $p*p > 20$, so every remaining candidate is prime; no further marking is needed.

4 Prime range queries: turn flags into counts

For many queries $[l, r]$, build a prefix count after the sieve:

```
pref[0] = 0
pref[i] = pref[i-1] + (1 if is_prime[i] else 0)
answer(l, r) = pref[r] - pref[l-1]
```

The subtraction removes exactly the primes before l . In particular, $l = 1$ is safe because $\text{pref}[0]$ exists.

4.1 Trace: a prefix answer

For $N=10$, the prefix values at selected indices are:

i	primes in $[1, i]$	pref[i]
1	none	0
2	2	1
5	2, 3, 5	3
10	2, 3, 5, 7	4

Therefore the number of primes in $[4, 10]$ is $\text{pref}[10] - \text{pref}[3] = 4 - 2 = 2$: they are 5 and 7.

5 Smallest prime factors

An SPF table stores the smallest prime dividing every $x \geq 2$. Initialize `spf[x]=x`; an untouched entry is therefore prime. When processing a prime p , write p into only previously untouched multiples.

```
spf[x] = x for all x
for p from 2 while p*p <= N:
    if spf[p] == p:                # p is prime
        for x from p*p to N step p:
            if spf[x] == x:
                spf[x] = p
```

The important operation is not “look up all factors.” It is repeatedly take a factor that is guaranteed to divide the remaining value, divide all its copies out, and continue. The number 1 has an empty factorization; handle it before the loop.

5.1 Trace: factor 360

remaining	chosen SPF	exponent so far
360	2	2^3 , remaining 45
45	3	3^2 , remaining 5
5	5	5^1 , remaining 1

Thus $360 = 2^3 * 3^2 * 5$. It has three distinct prime factors, six prime factors with multiplicity, and largest prime factor 5.

6 Divisor counts from exponents

If $x = p_1^{e_1} * p_2^{e_2} * \dots * p_k^{e_k}$, every positive divisor independently chooses one exponent in $0..e_i$ for each prime. Hence:

$$\tau(x) = (e_1+1)(e_2+1)\dots(e_k+1)$$

For the 360 trace, the answer is $(3+1)(2+1)(1+1)=24$. Do not add the exponents: addition gives the number of prime factors with multiplicity, not the number of divisors.

7 Implementation notes

C++. Use `vector<bool>` or `vector<char>` for prime flags and `vector<int>` for `spf`. Make the loop variable in `p*p` tests `long long`; that habit avoids overflow when a later problem raises the limit.

Python. A list of booleans/integers of length about one million is suitable. Keep the explicit sieve loop for clarity. Use `sys.stdin.buffer.read()` for the large query counts in the local exercises.

8 Wrong turns to reject early

Treating 1 as prime.

Using trial division separately for 200,000 queries.

Starting every sieve inner loop at $2 \cdot p$ without understanding the duplicated work.

Forgetting `pref[l-1]` when $l=1$.

Using `spf[x]` once and forgetting repeated copies of the same prime.

Returning zero divisors for $x=1$; its only divisor is 1.

9 Formative checks

1. On paper, sieve through 30. Which multiples are first marked when processing 5, and why is 10 not revisited?
2. Build the prefix counts for primes up to 12; use them to answer $[1,1]$, $[2,2]$, and $[8,12]$.
3. Factor 84 with SPF and derive its divisor count. Expected factorization: $2^2 * 3 * 7$; expected divisor count: 12.
4. Before coding B, state what each output component should be for $x=1$, a prime, and a prime power. This catches most factor-loop errors.

10 Local judge workflow

Run your implementation against fixed and randomized tests with:

```
CP_TARGET=solution python3 sections/31_sieve_primes_factorization/check.py
```

The generators compare small SPF-derived results with slow trial division. When debugging your own solution, first print the factor blocks for 1, a prime, a prime power, and a product of several primes.

11 Submission checklist

1. Is the precomputation limit at least the largest input value?
2. Are 0 and 1 non-prime?
3. Does every range query use `pref[r] - pref[l-1]`?
4. Does the factor loop remove all copies of the current prime?
5. Does `x=1` get its special, mathematically correct answer?

12 Exercises

Attempt A after the prefix trace, B after the factor 360 trace, and C after the divisor formula. For each exercise, write the chosen precomputation and its time/memory cost before opening the editor.